
Formatting Instructions for TMLR Journal Submissions

Student number: 22033917

GitHub repository: <https://github.com/UCL-ELEC0136/10-final-assessment-xzzc666>

Abstract

1 Introduction

2 System overview

3 Data discovery and acquisition

3.1 Data Sources and Usage Protocols

This project uses **FRED** (Federal Reserve Economic Data) as the primary data source. **FRED** provides a large collection of authoritative macroeconomic and financial time-series data and supports programmatic access through a free API key.

According to the official *FRED API Terms of Use*?, the free API may be used for academic research and non-commercial purposes. In this project, all retrieved data are used exclusively for model training and analysis. The original data are neither shared nor redistributed, and the usage fully complies with the relevant terms and conditions.

In addition, the official **FRED** website provides comprehensive API documentation? to guide standardized access to economic and financial time-series data. Since the data acquisition process in this project is based on indicator (series) identifiers, the *series observations* endpoint in the **FRED** API documentation is primarily referenced?. In the implementation, HTTP requests are sent using Python's **requests** library, with the parameters **api_key**, **file_type**, **series_id**, **realtime_start**, and **realtime_end** used to control the type and time range of the retrieved time series. This design enables an automated and fully reproducible data collection pipeline.

3.2 Dataset Selection

Table 1 summarises the data collected in this project and the rationale for selecting each data source. Due to space limitations, the full table is reported in *Appendix*.

In total, **31 distinct time-series datasets** are incorporated in this study. These datasets are organised into multiple economically meaningful categories, including the *target market*, *auxiliary equity markets*, *volatility and systemic risk*, *credit spreads*, *interest rates and term structure*, *inflation*, *liquidity and monetary conditions*, *labour market*, *economic activity*, *housing market*, and *consumption and sentiment*. This structured design enables the model to characterise market behaviour from complementary macro-financial perspectives rather than relying on a single information source.

The dataset spans the period from **January 1, 2015** to **December 31, 2025**, covering multiple market regimes and macroeconomic cycles.

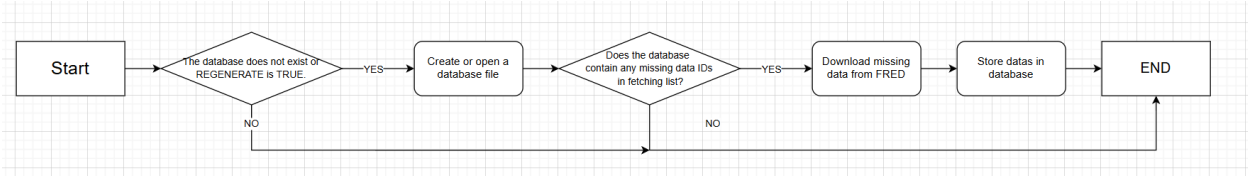


Figure 1: Data acquisition and storage workflow

In the following training stage shows that using a limited number of input variables led to pronounced overfitting, particularly under high-capacity model configurations. Therefore, a relatively large set of macroeconomic and financial indicators is intentionally adopted.

Although incorporating numerous input variables increases the dimensionality of the feature space, this challenge is addressed in later stages (*Representation Learning, Feature Design, and Impact Analysis*) through Principal Component Analysis (PCA). PCA enables effective dimensionality reduction while preserving the dominant information structure, thereby mitigating the negative impact of dimensionality growth on model training and generalisation performance.

3.3 Data storage and log writing

Data storage is necessary because repeatedly retrieving identical historical data through APIs not only leads to access rate limitations and increased server load, but also significantly raises the time cost of data acquisition. During model training and multiple experimental runs, storing all data in a unified local database effectively avoids redundant requests, improves overall computational efficiency, and ensures data consistency as well as the reproducibility of experimental results.

In this project, an SQLite database is adopted as the data storage solution. SQLite is a lightweight local database natively supported by Python, offering advantages such as offline operation, no account configuration, and simple deployment. Since the data in this project only need to be fully acquired once and stored for long-term use—without requirements for data sharing, high-concurrency access, or frequent write operations—a local database solution is sufficient to meet all project needs. Furthermore, SQL is selected instead of a NoSQL database to ensure strict temporal alignment across different macroeconomic and financial indicators, thereby avoiding data mismatch issues caused by flexible or non-uniform field structures. This design improves the structural stability and overall reliability of the dataset.

The overall execution logic is illustrated in Figure 1. A hyperparameter **REGENERATE** is introduced to control whether the raw feature data should be regenerated. When **REGENERATE** is set to **true** or when the database file does not exist, the program automatically detects missing data series and sends API requests only for the missing components. The complete dataset is then stored in the **data_raw.db** database located in the **data** directory.

In addition, to facilitate runtime monitoring and the identification of potential errors, a **WriteLog** function is implemented. This function outputs progress information and error messages to the terminal while simultaneously recording them in the **data_fetch.log** file under the **data** directory, thereby enhancing the traceability and debugging efficiency of the data collection pipeline.

-
- 4 Object model, data validation, and storage
 - 5 Dataset construction and sampling
 - 6 Problem formalisation
 - 7 Representation learning, feature design and impact analysis
 - 8 Training and evaluation protocol
 - 9 References
 - 10 Appendix

Table 1: Dataset selected for model training and evaluation.

CATEGORY	INDICATORS	REASON
Target market	SP500	Serves as the prediction target, providing the reference equity dynamics for supervised learning
Auxiliary equity markets	DJIA, NASDAQCOM	Capture cross-market co-movement and sectoral divergence that may provide contextual signals for SP500 price dynamics
Volatility and systemic risk	VIXCLS, TEDRATE, STLFSI4	Provide forward-looking information on market uncertainty, which may influence risk premia and short-term SP500 fluctuations
Credit spreads	BAMLH0A0HYM2, BAMLC0A0CM, BAMLC0A1CAAA, BAMLC0A4CBBB	Reflect changes in financing conditions and investor risk appetite that can affect equity valuation and market drawdowns
Interest rates and term structure	GS3M, GS2, GS10, GS30, T10Y2Y, T10Y3M	Encode monetary policy stance and yield curve expectations that influence discount rates applied to SP500 constituents
Inflation	CPIAUCSL, CPILFESL, PCEPI	Capture inflation dynamics that shape policy expectations and indirectly impact equity valuation levels
Liquidity and monetary conditions	M2SL, WALCL	Represent liquidity expansion or contraction that may amplify or suppress SP500 market momentum
Labour market	UNRATE, PAYEMS	Indicate employment conditions affecting household income, consumption capacity, and earnings expectations of SP500 firms
Economic activity	INDPRO, TCU, DGORDER	Describe real economic momentum that may influence corporate profitability and medium-term equity trends
Housing market	CSUSHPISA, HOUST, PERMIT	Reflect interest-rate-sensitive economic sectors that often respond early to policy shifts impacting broader equity markets
Consumption and sentiment	UMCSENT, RSAFS	Provide behavioural signals of household expectations that may affect equity demand and short-term market sentiment